Tecnicatura Superior en Desarrollo de Software

Módulo 09 — Host de escritorio: `Composition Root`, Configuración y Ciclo de Vida

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026

¿Qué es el host y el composition root?

Host y Composition Root	El programa ejecutable que reúne todas las capas y las pone a funcionar	Capa: la más externa (Frameworks y Drivers)

Las capas de Clean Architecture (Domain, Application, InterfaceAdapter, Infrastructure) son **bibliotecas**: contienen clases, pero ninguna sabe arrancar sola. El **host** es el programa ejecutable que las reúne. El **composition root** (raíz de composición) es el único lugar de ese programa donde se decide qué clase concreta usa cada interfaz, es decir, donde se "arma" la aplicación.

Qué hace:

- Lee la **configuración** (archivos `appsettings`)
- Arma el **contenedor** de inyección de dependencias
- Controla el **ciclo de vida** de la aplicación
- Muestra la **primera pantalla**

Hasta este módulo cada capa se registró en su propio método (`AddUseCases`, `AddPresenters`, `AddInfrastructure`...) pero nadie los llamaba. El host es quien los llama a todos, una sola vez y en un solo lugar.

Analogía del mundo real

Pensá en el **tablero eléctrico** de un edificio. Los planos de cada departamento dicen qué artefactos tiene (cocina, luces, heladera), pero ninguno sabe de dónde viene la electricidad: cada uno solo tiene un enchufe. El tablero es el único lugar donde se decide qué cable alimenta a qué departamento, y donde se conecta la red de la compañía.

Si mañana cambia la compañía eléctrica, **solo se toca el tablero**: ningún departamento se entera. Y si un departamento tuviera su propio cable tirado hacia la calle, el edificio sería imposible de mantener. El host es ese tablero: los enchufes son las interfaces que declaran las capas, y los cables son los registros en el contenedor.

Características esenciales

1. Conoce a todas las capas; ninguna lo conoce a él

Es el único proyecto que referencia a InterfaceAdapter y a Infrastructure al mismo tiempo. La regla de dependencias sigue intacta: las flechas apuntan hacia adentro, y el host es el punto más externo, del que nadie depende.

2. Es el único lugar donde se escribe `new` de infraestructura

Las capas internas piden interfaces. Qué clase las implementa se decide acá, en un solo archivo. Cambiar de base de datos o de tecnología de pantalla es cambiar el composition root, no las capas.

3. No contiene lógica de negocio

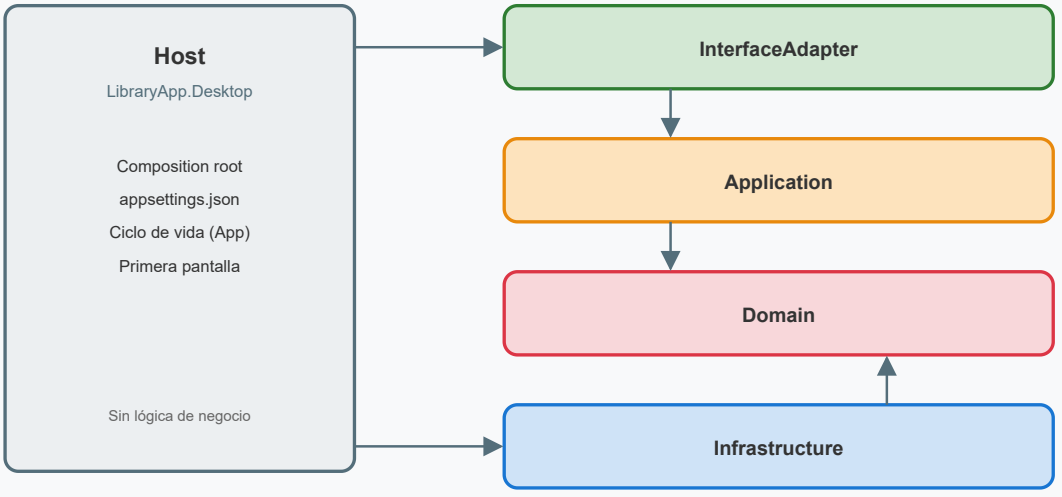
Si el host calcula, valida o decide algo del negocio, esa lógica quedó en el lugar equivocado. Su trabajo es conectar y arrancar.

4. Es el dueño de la configuración y del ciclo de vida

Lee de dónde está la base de datos, decide cuándo nace y cuándo muere la aplicación, y libera los recursos al cerrar.

¿Cómo encaja en la arquitectura?

El host es el anillo más externo. Referencia solamente a InterfaceAdapter e Infrastructure: las demás capas llegan a él por *transitividad* (si el proyecto A referencia a B y B referencia a C, A puede usar C sin declararlo).



Las flechas significan "referencia a". El host referencia a InterfaceAdapter e Infrastructure y, a través de ellas, llega a todas las capas.

Nadie referencia al host. Por eso se puede reemplazar: una aplicación de escritorio, una consola o una aplicación web serían distintos hosts sobre las mismas capas.

Las referencias y el paquete del proyecto host

Igual que en los módulos anteriores, nada se importa solo: todo lo que el host usa está declarado en su archivo de proyecto. Este es el de una aplicación de escritorio de biblioteca:

```xml
<Project Sdk="Microsoft.NET.Sdk">

  <ItemGroup>
    <ProjectReference Include="..\InterfaceAdapter\LibraryApp.InterfaceAdapter.csproj" />
    <ProjectReference Include="..\Infrastructure\LibraryApp.Infrastructure.csproj" />
  </ItemGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.Extensions.Hosting" Version="9.0.20" />
  </ItemGroup>

  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net9.0-windows</TargetFramework>
    <UseWPF>true</UseWPF>
  </PropertyGroup>

</Project>
```

- `OutputType` = `WinExe`: el resultado es un ejecutable de Windows sin consola, no una biblioteca (`.dll`).
- `net9.0-windows` y `UseWPF`: WPF usa APIs exclusivas de Windows, por eso el sufijo `-windows` , y `UseWPF` activa el compilador de XAML.
- **Dos referencias a proyecto:** InterfaceAdapter e Infrastructure. Application y Domain llegan por transitividad.
- `Microsoft.Extensions.Hosting`: el paquete NuGet con el *host genérico*: la clase `Host` , que trae configuración, inyección de dependencias y logging ya integrados. Se instala con `dotnet add package` y, como todo paquete, con versión exacta de la misma rama que el framework (9.x para `net9.0`).

El host genérico: Microsoft.Extensions.Hosting

Una aplicación moderna de .NET necesita siempre las mismas tres cosas: leer su configuración, tener un contenedor de inyección de dependencias y registrar mensajes de diagnóstico. En lugar de armarlas a mano, el **host genérico** las trae ya integradas. Se construye con `Host.CreateDefaultBuilder()` :

```
var host = Host.CreateDefaultBuilder()
```

```
    .UseContentRoot(AppContext.BaseDirectory)
    .ConfigureServices((context, services) => services.AddLibrary(context.Configu
ration))
    .Build();
```

- `CreateDefaultBuilder()` crea el constructor con la configuración estándar: carga `appsettings.json`, el archivo del entorno, las variables de entorno y los argumentos de línea de comandos si los hay, y registra el contenedor y el logging.
- `UseContentRoot(...)` fija la carpeta donde se buscan los archivos de configuración. Por defecto es la carpeta *desde la que se lanzó el programa*, que no siempre es donde está el ejecutable; `AppContext.BaseDirectory` es la carpeta del ejecutable.
- `ConfigureServices(...)` recibe la configuración ya cargada y la colección de servicios: es el punto de entrada al composition root.
- `Build()` construye el host. Después se arranca con `StartAsync()` y se detiene con `StopAsync()`.

La configuración: appsettings.json

Lo que cambia según dónde se ejecuta la aplicación (la dirección de la base de datos, un usuario, una contraseña) **no se escribe en el código**: se lee de un archivo de configuración, para poder cambiarlo sin recompilar y sin dejar secretos en el repositorio. El estándar es `appsettings.json`:

```
{
  "ConnectionStrings": {
    "LibraryDb": "Server=(localdb)\\MSSQLLocalDB;Database=Library;Trusted_Connect
ion=True;"
  }
}
```

Es un árbol de claves y valores (la barra invertida se escribe doble en JSON porque es el carácter de escape). La sección `ConnectionStrings` es una convención: el host sabe leerla con `configuration.GetConnectionString("L ibraryDb")`. Para que el archivo llegue al ejecutable hay que pedir que se copie a la carpeta de salida con `CopyToOutputDirectory` en el proyecto.

Una buena práctica es **fallar al arrancar si falta un dato obligatorio**, con un mensaje que diga qué falta y dónde, en lugar de dejar que el error aparezca más tarde en un lugar lejano:

```
var connectionString = configuration.GetConnectionString("LibraryDb")
    ?? throw new InvalidOperationException(
        "Falta la cadena de conexión 'LibraryDb' en appsettings.json.");
```

Más fuentes de configuración: el archivo no es la única fuente. Las variables de entorno y los archivos por entorno pueden *pisar* sus valores, y existen los secretos de usuario para desarrollo. El orden exacto y cómo usarlos está en el **anexo 09a**.

Tiempos de vida en una aplicación de escritorio

El contenedor ofrece tres tiempos de vida: **Singleton** (una instancia para toda la aplicación), **Scoped** (una instancia por "unidad de trabajo") y **Transient** (una instancia nueva cada vez). El anexo 04c los explica en detalle. Lo que cambia en el escritorio es quién define esa "unidad de trabajo":

- En una **aplicación web**, cada pedido HTTP abre un scope automáticamente.
- En una **aplicación de escritorio**, **nadie abre scopes por vos**. Hay que abrirlos explícitamente.

Esto importa porque el `DbContext` de Entity Framework está registrado como Scoped, y una ventana o un servicio de vida larga (Singleton) **no puede recibirlo por constructor**: el contenedor lo rechaza con *"Cannot consume scoped service ... from singleton"*. La solución es recibir la *fábrica de scopes* y abrir uno por operación:

```
public sealed class DatabaseHealthCheck
{
    private readonly IServiceScopeFactory _scopeFactory;

    public DatabaseHealthCheck(IServiceScopeFactory scopeFactory)
    {
        _scopeFactory = scopeFactory;
    }

    public async Task<bool> CanConnectAsync()
```

```
    {
        // se abre un scope para esta operación y se cierra al terminar
        using var scope = _scopeFactory.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<LibraryDbContext>
();

        return await context.Database.CanConnectAsync();
    }
}
```

Regla práctica: un servicio de vida larga que necesita algo de vida corta recibe `IServiceScopeFactory` y abre un scope por operación. En el módulo siguiente, los componentes de Blazor hacen lo mismo con `OwningComponentBase`.

Validar la composición: ValidateOnBuild y ValidateScopes

Por defecto el contenedor es perezoso: no comprueba nada hasta que alguien pide un servicio. Un servicio sin registrar o un Scoped dentro de un Singleton pueden pasar desapercibidos hasta que el usuario llega a esa pantalla. Dos opciones al construir el contenedor lo evitan, haciendo que **los errores de composición aparezcan al arrancar**:

```
var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,   // cada servicio registrado debe poder construirse
    ValidateScopes = true     // ningún Singleton puede depender de un Scoped
});
```

Los mensajes dicen exactamente qué falla. Por ejemplo, si se olvidó registrar una dependencia:

```
Unable to resolve service for type 'IUpdateBookOutputPort'
while attempting to activate 'UpdateBookUseCase'.
```

Y si un Singleton pide un Scoped:

```
Cannot consume scoped service 'LibraryDbContext' from singleton 'DatabaseHealthCh
eck'.
```

Conviene que una **prueba automática** construya el contenedor del composition root con estas dos opciones: así cada cambio que rompa la composición se detecta al correr las pruebas, sin abrir la aplicación.

El ciclo de vida de una aplicación WPF

Una aplicación WPF tiene una clase `App` (con su par `App.xaml`) que representa a todo el programa. Por defecto la plantilla incluye en `App.xaml` una línea `StartupUri="MainWindow.xaml"`, que le dice a WPF "creá esta ventana al arrancar". WPF la crea con `new`, **salteándose el contenedor**: la ventana no recibiría sus dependencias por constructor. Con un host genérico se quita esa línea y se muestra la ventana desde el código, resolviéndola del contenedor:

```
public partial class App : System.Windows.Application
{
    private readonly IHost _host;

    public App()
    {
        _host = Host.CreateDefaultBuilder()
            .UseContentRoot(AppContext.BaseDirectory)
            .ConfigureServices((context, services) => services.AddLibrary(context.Configuration))
            .Build();
    }

    protected override async void OnStartup(StartupEventArgs e)
    {
        await _host.StartAsync();

        // la ventana viene del contenedor: recibe sus dependencias por constructor
        _host.Services.GetRequiredService<MainWindow>().Show();

        base.OnStartup(e);
    }

    protected override async void OnExit(ExitEventArgs e)
    {
        await _host.StopAsync();
        _host.Dispose();

        base.OnExit(e);
```

```
    }
}
```

- **El constructor** construye el host (la configuración y el contenedor).
- `OnStartup` arranca el host y muestra la primera ventana.
- `OnExit` detiene el host y libera sus recursos.

Un detalle del nombre: la clase base se escribe `System.Windows.Application` completo. Cuando el proyecto host referencia a un proyecto cuyo namespace se llama `LibraryApp.Application`, el nombre corto `Application` pasa a significar ese espacio de nombres y el compilador falla con *"'Application' es espacio de nombres pero se usa como tipo"*.

Errores comunes

ERROR COMUN

Crear la ventana con `new`, saltándose el contenedor:

```
// en App.xaml: StartupUri="MainWindow.xaml"
// o en código:
new MainWindow().Show();   // no recibe DatabaseHealthCheck
```

La ventana no recibe sus dependencias y el programa falla al abrirse.

BUENA PRACTICA

Resolverla del contenedor:

```
_host.Services.GetRequiredService<MainWindow>().Show();
```

ERROR COMUN

Escribir la cadena de conexión en el código:

```
services.AddDbContext<LibraryDbContext>(options =>
    options.UseSqlServer("Server=prod;Database=Library;User Id=sa;Password=1234;"));
```

La contraseña queda en el repositorio y cambiar de servidor obliga a recompilar.

BUENA PRACTICA

Leerla de la configuración:

```
var connectionString = configuration.GetConnectionString("LibraryDb")
    ?? throw new InvalidOperationException("Falta la cadena de conexión 'LibraryDb'.");
services.AddInfrastructure(connectionString);
```

ERROR COMUN

Un Singleton que recibe un servicio Scoped:

```
public sealed class DatabaseHealthCheck
{
    public DatabaseHealthCheck(LibraryDbContext context) { }   // Scoped dentro de un Singleton
}
```

Con `ValidateScopes` el contenedor lo rechaza; sin esa opción el contexto queda vivo toda la aplicación y se corrompe.

BUENA PRACTICA

Recibir la fábrica de scopes y abrir uno por operación:

```
public DatabaseHealthCheck(IServiceScopeFactory scopeFactory) { /* ... */ }
```

ERROR COMUN

No validar la composición:

```
var provider = services.BuildServiceProvider();   // sin validación
// un servicio sin registrar se descubre recién cuando el usuario llega a es
a pantalla
```

BUENA PRACTICA

Validar al construir y en una prueba automática:

```
var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
```

Resumen

El host es el programa que reúne las capas y las pone a correr.

Referencia a InterfaceAdapter e Infrastructure (las demás llegan por transitividad), lee su configuración de `appsettings.json`, y en un único lugar, el **composition root**, registra todas las capas en el contenedor. Como en el escritorio nadie abre scopes por vos, los servicios de vida larga reciben `IServiceScopeFactory`. La ventana principal se resuelve del contenedor, y una prueba automática con `ValidateOnBuild` y `ValidateScopes` garantiza que la composición no se rompe.

Resultado: cambiar de base de datos o de tecnología de pantalla es cambiar el host, no las capas.